

2. What is Geometric Transformation

A geometric transformation is a mathematical operation altering the spatial position, orientation, scale, or shape of an object without changing its underlying data values. It changes *where* the image information is located ($I(x, y) \rightarrow I(x', y')$), whereas a photometric transformation changes *what* the pixel looks like (e.g., brightness or color: $I(x, y) \rightarrow I'(x', y')$).

3. The Image Coordinate System

Unlike a standard mathematical Cartesian plane that places the origin (0,0) at the bottom-left, computer vision images (including OpenCV) place the origin at the top-left corner. The x-axis extends to the right, and the y-axis extends downward.

4. 2D vs 3D Transformation

- **2D Transformation:** Operates on an image plane, photograph, or document. Typical representations are 2×3 matrices for affine warping or 3×3 matrices for homogeneous coordinates.
- **3D Transformation:** Operates on 3D world space coordinates $(x, y, z) \rightarrow (x', y', z')$ using 4×4 homogeneous representations, common in robotics and 3D graphics.

5. Transformation Matrix

A transformation matrix mathematically maps how to warp, rotate, scale, or shear a 2D space. To visualize it, read a 2×2 matrix vertically: the first column vector is the new location of the X-axis, and the second column is the new location of the Y-axis.

- **Variable a:** Controls Horizontal Scaling (values > 1 stretch; negatives flip).
- **Variable c:** Controls Vertical Shearing (tilts vertical grid lines).
- **Variable b:** Controls Horizontal Shearing (tilts horizontal grid lines).
- **Variable d:** Controls Vertical Scaling.

6. 2D Geometric Transformations

6.1 Affine Transformation

An affine transformation preserves collinearity (points on a line remain on a straight line) and parallelism (parallel lines remain parallel). It is mathematically expressed as a linear transformation plus a translation: $p' = Ap + b$. While the coordinate grid can be skewed into parallelograms, lines never curve or break, and the ratio of distances between points is maintained.

6.1.1 Linear Transformation

A linear transformation is a subset of affine geometry that relies strictly on matrix multiplication ($p' = Ap$), omitting translation.

- **Scaling:** Resizes the shape along the axes.
- **Rotation:** Tilts the space using sine and cosine trigonometric functions to perfectly balance both axes so they remain perpendicular.
- **Reflection:** Flips the geometry like a mirror across an axis using a negative diagonal value.
- **Shearing:** Skews the grid by adding a fraction of one coordinate to another.
- **The Limitation (Anchored Origin):** Because multiplying any matrix by the origin (0,0) mathematically results in (0,0), a linear transformation cannot slide (translate) an object across the grid. The origin is permanently anchored.
- **The Solution (Homogeneous Coordinates):** To bypass this without breaking hardware processing pipelines, a dummy "1" is added to coordinates, making them $(x, y, 1)$. Upgrading to a 3×3 matrix folds translation directly into continuous matrix multiplication, allowing GPUs to process scale, rotation, and translation simultaneously.

6.2 Projective Transformations (Homography)

Projective transformations break the strict affine rule of parallelism, simulating how human eyes and cameras perceive 3D depth by allowing parallel lines to converge.

- **The Mathematical Core:** Uses a full 3×3 matrix. To find final 2D coordinates, the resulting vector must be normalized by dividing by the third scaling coordinate (w').
- **The Denominator Effect:** In an affine transformation, the denominator is always 1. In a projective transformation, the denominator fluctuates based on the x and y values, shrinking coordinates as points move "further away" to generate depth.
- **Real-World Modeling:** Accurately corrects planar alignment (like fixing a photograph of a document taken from an angle) and simulates perspective distortion (converging roads).

Geometric Transformation Notebook

1. Loading and Displaying Base Image

```
Python
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")
if image is None:
    print("Error: image.jpg not found")
else:
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert BGR to RGB for
    Matplotlib[cite: 51]

plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

2. Affine → Linear Rotation

```
Python
height, width = image.shape[:2]
center = (width // 2, height // 2)
angle = 45
scale = 1.0 # 1 means no scaling[cite: 52]
```

```
M = cv2.getRotationMatrix2D(center, angle, scale) # Create rotation matrix[cite: 52]
rotated = cv2.warpAffine(image, M, (width, height)) # Apply rotation[cite: 52]
```

3. Affine → Linear Scale

```
Python
scale_x = 3
scale_y = 3
```

```
scaled = cv2.resize(image, None, fx=scale_x, fy=scale_y, interpolation=cv2.INTER_LINEAR) #
Apply scaling[cite: 53]
```

4. Affine → Linear Shear

```
Python
kx = 0.5 # Shearing factor[cite: 55]
```

```
# X-shear matrix[cite: 55]
M = np.float32([
    [1, kx, 0],
    [0, 1, 0]
])
```

```
new_width = int(width + abs(kx) * height) # Calculate new width[cite: 55]
sheared = cv2.warpAffine(image, M, (new_width, height))
```

5. Affine → Translation

```
Python
tx = 100
ty = 50
```

```
# Translation matrix[cite: 56]
```

```
M = np.float32([  
    [1, 0, tx],  
    [0, 1, ty]  
])
```

```
translated = cv2.warpAffine(image, M, (width, height))
```

6. Rigid Transformation (Rotation + Uniform Scaling + Translation)

Python

```
angle = 30
```

```
scale = 1.0 # Scale MUST be 1.0 for rigid transformation[cite: 58]
```

```
M = cv2.getRotationMatrix2D(center, angle, scale)
```

```
# Add translation[cite: 58]
```

```
M[0, 2] += 100
```

```
M[1, 2] += 50
```

```
rigid = cv2.warpAffine(image, M, (width, height))
```

7. Projective → Perspective

Python

```
# Original corners[cite: 59]
```

```
src_points = np.float32([  
    [0, 0],  
    [width - 1, 0],  
    [width - 1, height - 1],  
    [0, height - 1]  
])
```

```
# New perspective positions[cite: 59, 60]
```

```
dst_points = np.float32([  
    [50, 0],  
    [100, 50],  
    [width - 50, height - 80],  
    [50, height - 20]  
])
```

```
# Homography matrix[cite: 60]
```

```
M = cv2.getPerspectiveTransform(src_points, dst_points)
```

```
# Apply perspective[cite: 60]  
perspective = cv2.warpPerspective(image, M, (width, height))
```